High Level Design Document — Drone Mapper

Advanced Topics in Software Engineering — TAU 2026B, Assignment 1

May 2026

---

1. Introduction

This document presents the High Level Design (HLD) of the Drone Mapper system, a simulation-based autonomous drone that scans a three-dimensional mission volume and produces a labelled occupancy map. The system is implemented in C++ and is driven by configuration files that define the drone's physical constraints, the mission boundary polygon, and the map resolution. The main deliverables are an occupancy map file and a performance score that measures how accurately and completely the drone mapped the environment.

The HLD covers the overall architecture, the key design decisions and the alternatives that were considered, and the approach used to validate correctness. UML class and sequence diagrams are provided alongside the textual explanations.

2. System Architecture

2.1 Layered Structure

The system is organised into four logical layers, each with a clearly scoped responsibility.

Layer 1 — Algorithm Control (MappingAlgorithm): The top layer contains the MappingAlgorithm class, which is the heart of the system and orchestrates the entire scanning process. It decides where the drone should fly next, when to scan, and when to move to a new height layer. Because it must manage a non-trivial amount of runtime state — current height level, heading, lidar beam bounds, output resolution, and the BFS frontier queue — it is the most data-rich class in the design. The two most important members are the Drone reference, which is the actuator through which all movement and sensing commands are issued, and m_frontier, a queue of three-dimensional grid cells that represents the drone's pending work list. As new unvisited neighbours are discovered

during scanning, they are pushed onto this queue; the main loop pops from it until no cells remain.

Layer 2 — Drone Facade (Drone): The Drone class represents the physical drone and exposes a small set of primitive commands: Rotate, Advance, Elevate, and Scan. It holds references to the four hardware interfaces (ILidarSensor, IPositionSensor, IMovementDriver, IMapBuilder) but contains no decision logic of its own — it simply forwards commands to the appropriate interface. This separation means MappingAlgorithm reasons at the level of "go to cell (x, y, z)" while Drone handles the lower-level translation into movement and sensing calls. The key design principle here is that the algorithm layer and the execution layer are decoupled: replacing the drone's internals (for example, switching from a simulated lidar to a real one) does not require any change to MappingAlgorithm.

Layer 3 — Simulation Mocks (MockMovementDriver, MockLidarSensor, MockPositionSensor, DenseVoxelMap): The third layer contains concrete implementations of the four hardware interfaces that simulate a physical environment. All three sensor and driver mocks share a single SimulationState object via shared_ptr, ensuring they always observe a consistent drone pose. DenseVoxelMap provides a high-resolution voxel representation of the obstacle map to support accurate ray-marching during lidar simulation.

Layer 4 — Mapping and Output (MapBuilder, MapIO, ScoreEvaluator, ConfigParser, ErrorLogger): The bottom layer maintains the output map and handles all file I/O. Each utility class has exactly one responsibility: MapIO only reads and writes map files, ConfigParser only parses configuration text, ErrorLogger only records error messages, and ScoreEvaluator only computes the final performance index. This strict single-responsibility decomposition makes each class independently testable and easy to audit.

2.2 Separation of Concerns

A guiding principle throughout the design is the clear separation between three roles:

- Who decides where to fly: MappingAlgorithm, which holds the BFS frontier and the exploration strategy.
- Who executes the flight: Drone, which translates high-level commands into interface calls.
- Who remembers how the world looks: MapBuilder, which maintains the output occupancy grid and enforces boundary constraints.

None of these three components has any direct knowledge of the others' internal implementation. Communication flows downward through well-defined interfaces, making each layer substitutable without impacting the others.

2.3 Memory Management via Hashing

The output map (MapBuilder) uses an unordered hash map keyed on the three-dimensional grid coordinate (ix, iy, iz). This design choice means that memory consumption grows only in proportion to the number of cells the drone actually visits, rather than the total size of the mission volume. For large mission volumes this can be a significant saving. Search and insert operations remain O(1) on average. The hash key is the (x, y, z) integer triple, and a custom KeyHash functor combines the three fields using a standard seed-mixing pattern to avoid clustering.

3. Design Considerations

3.1 Interface-Based Decoupling

The mapping algorithm (MappingAlgorithm) and the drone facade (Drone) interact exclusively through five abstract interfaces: ILidarSensor, IPositionSensor, IMovementDriver, IMapBuilder, and IMap3D. Neither class holds a reference to any concrete implementation. This means the entire algorithmic layer is completely unaware that it is operating inside a simulation. Swapping the mock implementations for drivers that communicate with real hardware requires zero changes to the algorithm or drone code — only the wiring in main.cpp changes. This architectural choice aligns with the Dependency Inversion Principle and is the primary enabler of testability: unit tests can inject stub implementations of any interface without building the full simulation stack.

3.2 Shared SimulationState

All three sensor/driver mocks — MockPositionSensor, MockMovementDriver, and MockLidarSensor — share a single SimulationState struct via shared_ptr. The struct holds the drone's current Position3D and Orientation. When MockMovementDriver::Advance() updates the position, MockLidarSensor and MockPositionSensor immediately observe the new state without any explicit synchronisation, since there is only one object in memory. This design eliminates the risk of the three mocks having inconsistent views of the drone's pose, which is a common source of subtle simulation bugs when each mock maintains its own copy.

3.3 DenseVoxelMap Solid-Block Fill

The input map stores occupied cells at the mission grid resolution (5 cm per cell). The DenseVoxelMap, however, operates at 1 cm voxel resolution to support accurate ray-marching and per-centimetre collision checks. Without additional treatment, a 1 cm ray marching diagonally through a 5 cm grid could slip through the 4 cm gap between adjacent voxel centres, producing phantom empty readings through solid walls. The fix is to fill each occupied grid cell as a complete 5x5x5 cm solid block in the dense map. The grid resolution is detected automatically from the minimum coordinate step found in the parsed map, so the algorithm is not hard-coded to 5 cm and works correctly for any resolution that matches map_resolution_cm.

3.4 MapBuilder Boundary Enforcement

MapBuilder::Set() silently discards any write whose (x, y) coordinate falls outside the mission boundary polygon, or whose z falls outside [min_height, max_height]. MapBuilder::Get() returns NotReachable (-2) for out-of-bounds queries and NotMapped (-1) for in-bounds cells that have never been written. This cleanly partitions the output value space: the drone can never accidentally pollute the map with observations made near the boundary, and the caller never needs to check bounds before calling Get(). The classification logic is centralised in one place, making it trivial to audit.

3.5 BFS Exploration Strategy

The exploration algorithm is a breadth-first search over a discrete XY grid. At each BFS node the drone scans in all cardinal and diagonal directions and at multiple altitudes, then advances to the nearest unvisited neighbour. The BFS step size is computed as max(10, int(beam_max x 0.5)) cm, balancing coverage (large steps reduce total travel) with navigability (small enough to avoid overshooting obstacles). Multi-altitude sweeps at each node ensure full vertical coverage of the mission volume.

Alternative considered: depth-first search would explore one corridor completely before backtracking. BFS was preferred because it always processes the nearest unvisited cell first, minimising total travel distance and ensuring the drone terminates quickly in compact spaces. A random walk was also considered but rejected because the algorithm must be deterministic for score reproducibility.

3.6 Post-Mission Output Augmentation

The drone's mapping code only writes Occupied (1) or Empty (0) values during the mission. It never emits -1 or -2. After MappingAlgorithm::Run() returns, main.cpp iterates every cell in the ground-truth map and calls MapBuilder::Get(). Unscanned in-bounds cells return -1 (not mapped) and are appended to the output; out-of-bounds cells return -2 (not reachable) and are similarly appended. This separation keeps the drone code clean — it knows nothing about scoring semantics — and ensures the output file always contains an entry for every ground-truth cell.

4. Testing Approach

4.1 Scenario-Based Integration Tests

Three end-to-end scenarios exercise progressively more complex environments:

Scenario 1 — Open room: A 200x200x100 cm rectangular room with no interior obstacles. Serves as the baseline correctness test. Any missed cell is a pure algorithm failure unrelated to obstacle handling.

Scenario 2 — Cube obstacle: Same room with a solid cube in the interior. Verifies that the collision-avoidance logic prevents the drone from entering the obstacle and that the

BFS correctly routes around it.

Scenario 3 — Asymmetric polygon and interior walls: The mission boundary polygon covers only x >= 75 cm. Interior walls create narrow passages. This scenario validates boundary enforcement (cells at x < 75 cm receive -2), path-finding through tight corridors, and the solid-block fill of the DenseVoxelMap.

4.2 Visual Inspection

A Python/matplotlib script generates top-down slice images (z = 50 cm) for both the ground-truth map and the algorithm output, side-by-side, for each scenario. These PNG files are stored under scenario*/original_output/ and provide a quick sanity check: gaps in coverage are immediately visible as uncoloured cells inside the room boundary.

4.3 Score Validation

Each scenario has an expected score range validated against the reference output stored in original_output/map_output.txt. The scoring formula (+1 for correct, -0.5 for wrong, -0.5 for not-mapped, excluding -2 cells from the denominator) is applied by the ScoreEvaluator class and the result is printed to stdout. A regression is flagged if the score drops below the reference value.

4.4 Resolution Validation

main.cpp detects the map resolution from the minimum coordinate step found in map_input.txt and compares it to map_resolution_cm in the mission config. A mismatch causes an early exit with a clear error message, preventing silent scoring errors caused by mismatched grid assumptions.

4.5 Error Injection

ErrorLogger is tested by supplying malformed configuration files containing unknown keys and out-of-range values. The program must complete without crashing and must produce a non-empty input_errors.txt. This validates that the error-collection path is exercised and that config parsing is fault-tolerant.

4.6 Collision Safety

MockMovementDriver performs a collision check at every 1 cm step along the intended movement path. If any intermediate position overlaps an occupied voxel the move is rejected and MoveResult::CollisionDetected is returned. The algorithm was verified to never trigger a collision across all three scenarios, confirming that the BFS advance distance and obstacle-aware path selection keep the drone clear of walls at all times.

4.7 Boundary Correctness

In Scenario 3, all ground-truth cells with x < 75 cm are confirmed to receive output value -2 (not reachable). Cells whose (x, y) is inside the boundary polygon are structurally guaranteed never to receive -2 because MapBuilder::Get() returns -2 only for

out-of-bounds queries, and the polygon test is the single source of that classification.

4.8 Future: Unit Tests

The following GTest unit tests are planned for the next development phase:

- MapBuilder: Set/Get boundary conditions — writes just inside and just outside the polygon, height clipping, resolution snapping.
- DenseVoxelMap: solid-block coverage — confirm that every 1 cm voxel within a 5 cm cell is reported occupied after loading.
- ScoreEvaluator: all ground-truth x output value combinations — verify the formula produces the correct numerator, denominator, and final percentage.
- MappingAlgorithm: BFS path completeness — on a known small map, verify that every reachable cell is visited and no cell is visited twice.